

Exam 1 Review

Worked Problems

Recurrences

Solve $T(n) = 10 T(n/2) + O(n^2)$.

Q1: Solution

Master theorem form $a T(n/b) + O(n^d)$ with

$$a = 10, \quad b = 2, \quad d = 2.$$

Compare d against $\log_b a$:

$$\log_2 10 \approx 3.32 > 2 = d.$$

So $d < \log_b a$: the leaves dominate.

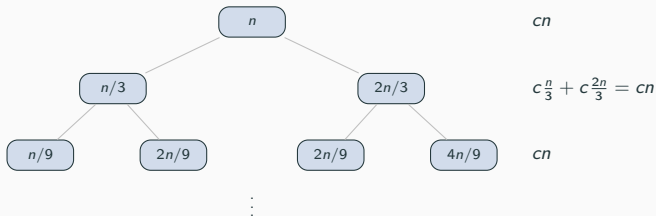
Answer

$$T(n) = O(n^{\log_2 10}) \approx O(n^{3.32})$$

Solve $T(n) = T(n/3) + T(2n/3) + O(n)$.

Q2: Solution

The Master theorem does **not** apply: the two subproblems have different sizes. Draw the recursion tree instead.



Since $\frac{1}{3} + \frac{2}{3} = 1$, every full level costs cn . The tree is lopsided: its shallowest leaf sits at depth $\log_3 n$ and its deepest at $\log_{3/2} n$, both $\Theta(\log n)$.

Answer

$$T(n) = \Theta(n \log n)$$

Solve $T(n) = T(n/10) + O(n^2)$.

Q3: Solution

Master theorem with

$$a = 1, \quad b = 10, \quad d = 2, \quad \log_{10} 1 = 0.$$

Since $d = 2 > 0 = \log_b a$, the **root dominates**.

Sanity check by unrolling: the work per level is

$$cn^2, \quad c\left(\frac{n}{10}\right)^2, \quad c\left(\frac{n}{100}\right)^2, \dots = cn^2 \left(1 + \frac{1}{100} + \frac{1}{100^2} + \dots\right) \leq \frac{100}{99} cn^2.$$

Answer

$$T(n) = O(n^2)$$

What is the worst-case running time recurrence for this algorithm?

```
Function foo(a[1:n], x)
  If n < 10:
    return a[1] + a[2] + ... + a[n] - x

  If a[n] > 10:
    y = 0
    for i = 1, 2, ..., n/3:
      y = y + a[i]
    return foo(a[1:n/3], y)
  else:
    return a[1] + a[2] + ... + a[n] + x * x
```

Q4: Solution

Work through the branches.

- **Base case** $n < 10$: a constant amount of work, $O(1)$.
- **else branch**: summing $a[1 \dots n]$ costs $O(n)$, no recursion.
- **if branch**: the loop runs $n/3$ times at $O(1)$ each, so $O(n)$, followed by **one** recursive call on $n/3$ elements.

The worst case is the branch that recurses:

Answer

$$T(n) = T(n/3) + O(n), \quad \text{which solves to } T(n) = O(n).$$

One subproblem of a third the size, so the work shrinks geometrically and the root dominates.

True or False

$3^n = O(2^n)$. True or false?

False.

For $3^n = O(2^n)$ we would need a constant c with $3^n \leq c \cdot 2^n$ for all large n , that is

$$\left(\frac{3}{2}\right)^n \leq c.$$

But $\lim_{n \rightarrow \infty} \frac{3^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{3}{2}\right)^n = \infty$, so no such constant exists.

In fact $3^n = \omega(2^n)$: different bases are **not** interchangeable.

$$1^{10} + 2^{10} + 3^{10} + \dots + n^{10} = \Theta(n^{10}).$$

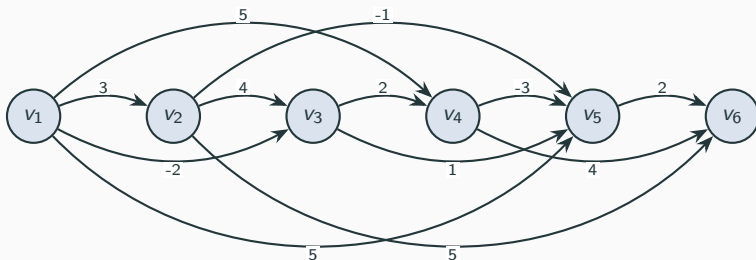
True or false?

False. It is $\Theta(n^{11})$.

Shortest Paths in a DAG

Q7

The vertices $v_1, \dots, v_6$ are in topological order, so every edge goes left to right. Edge lengths may be **negative**.



Compute the shortest path length from v_1 to each of $v_2, \dots, v_6$.

Q7: Solution

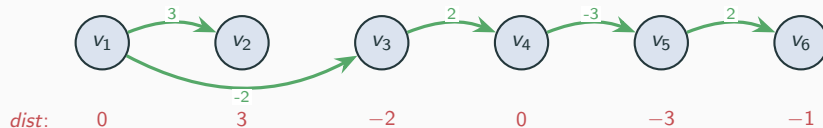
Fill left to right using $\text{dist}[v_j] = \min_{v_i \rightarrow v_j} (\text{dist}[v_i] + W[v_i, v_j])$.

- $\text{dist}[v_2] = 0 + 3 = 3$
- $\text{dist}[v_3] = \min(\underbrace{0}_{v_1} - 2, \underbrace{3}_{v_2} + 4) = -2$
- $\text{dist}[v_4] = \min(\underbrace{0}_{v_1} + 5, \underbrace{-2}_{v_3} + 2) = 0$
- $\text{dist}[v_5] = \min(\underbrace{0}_{v_1} + 5, \underbrace{3}_{v_2} - 1, \underbrace{-2}_{v_3} + 1, \underbrace{0}_{v_4} - 3) = -3$
- $\text{dist}[v_6] = \min(\underbrace{3}_{v_2} + 5, \underbrace{0}_{v_4} + 4, \underbrace{-3}_{v_5} + 2) = -1$

Answer

$\text{dist} = [0, 3, -2, 0, -3, -1]$ for $v_1, \dots, v_6$

Q7: The Shortest Path Tree



target	shortest path	
v_2	$v_1 \rightarrow v_2$	3
v_3	$v_1 \rightarrow v_3$	-2
v_4	$v_1 \rightarrow v_3 \rightarrow v_4$	0
v_5	$v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5$	-3
v_6	$v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6$	-1

Reading DP Tables

Let $K[i, c]$ be the maximum value using items $1, \dots, i$ with capacity c . Suppose

$$\begin{array}{llll} K[149, 400] = 910 & K[149, 280] = 700 & K[148, 400] = 880 & w[150] = 120 \\ K[149, 120] = 310 & K[149, 250] = 640 & K[100, 200] = 450 & v[150] = 260 \\ K[148, 280] = 690 & K[149, 300] = 760 & w[125] = 75 & v[125] = 140 \end{array}$$

What is $K[150, 400]$?

Q8: Solution

The recurrence looks only at row $i - 1$:

$$K[i, c] = \max \left(\underbrace{K[i - 1, c]}_{\text{skip item } i}, \underbrace{K[i - 1, c - w[i]] + v[i]}_{\text{take item } i} \right).$$

With $i = 150$, $c = 400$, $w[150] = 120$, $v[150] = 260$:

$$K[150, 400] = \max (K[149, 400], K[149, 400 - 120] + 260) = \max (910, 700 + 260).$$

Answer

$$K[150, 400] = \max(910, 960) = 960 \quad (\text{take item 150})$$

Everything else listed is a distractor: rows other than 149 and capacities other than 400 and 280 never appear.

Let $ED[i, j]$ be the edit distance between $A[1 \dots i]$ and $B[1 \dots j]$. Suppose $A[7] = T$ and $B[5] = T$, and

$$\begin{array}{llll} ED[6, 5] = 8 & ED[7, 4] = 6 & ED[6, 4] = 5 & ED[5, 5] = 9 \\ ED[7, 3] = 11 & ED[6, 6] = 7 & ED[5, 4] = 4 & ED[7, 6] = 9 \end{array}$$

What is $ED[7, 5]$?

Q9: Solution

$$ED[i, j] = \min \left(\underbrace{ED[i-1, j] + 1}_{\text{delete } A[i]}, \underbrace{ED[i, j-1] + 1}_{\text{insert } B[j]}, \underbrace{ED[i-1, j-1] + \text{diff}(i, j)}_{\text{substitute}} \right).$$

Since $A[7] = B[5] = \text{T}$, we have $\text{diff}(7, 5) = 0$. So

$$ED[7, 5] = \min \left(\underbrace{8+1}_{ED[6,5]}, \underbrace{6+1}_{ED[7,4]}, \underbrace{5+0}_{ED[6,4]} \right).$$

Answer

$$ED[7, 5] = \min(9, 7, 5) = 5 \quad (\text{characters match, so no edit})$$

Majority Element

Let L, R be a partition of $A[1 : n]$.

If x is the majority of L , then x is always the majority of A .

True or false?

False.

Counterexample. Take $L = [a, a]$ and $R = [b, b, b, b]$.



a occurs twice in L , and $2 > |L|/2 = 1$, so a is the majority of L .

But in A it occurs 2 times out of $n = 6$, and $2 \not> 3$. So a is not the majority of A .

Let L, R be a partition of $A[1 : n]$.

If x is the majority of L *and* the majority of R ,
then x is always the majority of A .

True or false?

Q11: Solution

True.

By assumption,

$$\text{count}(x, L) > \frac{|L|}{2} \quad \text{and} \quad \text{count}(x, R) > \frac{|R|}{2}.$$

Adding the two inequalities,

$$\text{count}(x, A) = \text{count}(x, L) + \text{count}(x, R) > \frac{|L| + |R|}{2} = \frac{n}{2}.$$

So x occurs strictly more than $n/2$ times in A . $\square$

Compare with Q10: one half is not enough, but both halves is.

Algorithm Design

Q12

A sorted array $A[1 : n]$ of **distinct** elements has been cyclically shifted k positions to the right. Find k in $O(\log n)$ time.



The last k elements have wrapped around to the front.

Q12: Solution

Key observation. The array is two sorted runs, and the **minimum** sits at index $k + 1$. So finding k means finding the minimum's index.

Compare $A[mid]$ against the **last** element $A[hi]$:

- $A[mid] > A[hi]$: the wrap point is to the *right*, so search $[mid + 1, hi]$.
- otherwise: mid is in the second run, so the minimum is at mid or to its left, search $[lo, mid]$.



Q12: Solution

```
# Index of the minimum inside A[lo:hi]
function min_index(A, lo, hi)
    if lo == hi
        return lo # base case: one element left
    end

    mid = div(lo + hi, 2)
    if A[mid] > A[hi]
        return min_index(A, mid + 1, hi) # wrap point is to the right
    else
        return min_index(A, lo, mid) # minimum is at mid or left of it
    end
end

find_shift(A) = min_index(A, 1, length(A)) - 1
```

Each call discards half the range, so $T(n) = T(n/2) + O(1) = O(\log n)$ by the Master theorem.

The 0-1 knapsack problem with values $v[1 : n]$, weights $w[1 : n]$, and capacity W , plus one extra constraint:

at most k items may be selected.

Design a dynamic programming algorithm that outputs the maximum achievable value.

Q13: Solution

Add a **third dimension** counting how many items we have used.

Definition

$K[i, c, j]$ = maximum value using items $1 \dots i$, capacity c , and at most j items.

The same two choices as before, but taking an item now also spends one of the j slots:

$$K[i, c, j] = \max \left(\underbrace{K[i-1, c, j]}_{\text{skip } i}, \underbrace{K[i-1, c - w[i], j-1] + v[i]}_{\text{take } i, \text{ if } w[i] \leq c, j \geq 1} \right).$$

Base cases: $K[0, c, j] = 0$ and $K[i, c, 0] = 0$.

Q13: Solution

The answer is $K[n, W, k]$.

Answer

$O(nWk)$ time and $O(nWk)$ space (reducible to $O(Wk)$ by keeping only row $i - 1$).

Q14: Vasya's Vacation

Vasya has n vacation days. For day i one of four things holds:

a_i	gym	contest
0	closed	no
1	closed	yes
2	open	no
3	open	yes

Each day he rests, writes the contest (if held), or does sport (if the gym is open). He will **not repeat the same activity on two consecutive days**.

Minimise the number of rest days.

Q14: Solution

The only thing that constrains tomorrow is **what he did today**. So make that the state.

Definition

$dp[i][s]$ = fewest rest days over days $1 \dots i$, where day i ended with activity $s \in \{\text{rest, contest, sport}\}$.

$$dp[i][\text{rest}] = 1 + \min (dp[i-1][\cdot])$$

$$dp[i][\text{contest}] = \min (dp[i-1][\text{rest}], dp[i-1][\text{sport}]) \quad \text{if } a_i \in \{1, 3\}$$

$$dp[i][\text{sport}] = \min (dp[i-1][\text{rest}], dp[i-1][\text{contest}]) \quad \text{if } a_i \in \{2, 3\}$$

Unavailable options are $+\infty$. Answer: $\min_s dp[n][s]$, in $O(n)$ time.

Q14: Example

$$a = [1, 3, 2, 0, 3, 3, 1]$$

a_i	1	3	2	0	3	3	1
plan	C	S	S	R	C	S	C

C = contest, S = sport, R = rest

Day 4 has $a_4 = 0$, so it is a forced rest. Everything else alternates legally.

Answer

Minimum rest days = 2

Q15: One Flip

Given $a_1, \dots, a_n$ with each $a_i \in \{0, 1\}$, choose indices $i \leq j$ and flip every a_k with $i \leq k \leq j$ (that is $x \mapsto 1 - x$).

You must make **exactly one** move. Maximise the number of ones.

a :

1	0	0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---

Q15: Solution

Flipping the range $[i, j]$ turns its zeros into ones and its ones into zeros, so

$$\text{ones after} = (\text{total ones}) - \text{ones in } [i, j] + \text{zeros in } [i, j].$$

Define

$$b_k = \begin{cases} +1 & \text{if } a_k = 0, \\ -1 & \text{if } a_k = 1. \end{cases}$$

Then the gain from flipping $[i, j]$ is exactly $\sum_{k=i}^j b_k$.

The answer is $(\text{total ones}) + (\text{maximum subarray sum of } b)$, over **non-empty** subarrays.

Kadane's algorithm gives this in $O(n)$.

Q15: Example

$a:$	1	0	0	1	0	0	1	0	1	0
$b:$	-1	+1	+1	-1	+1	+1	-1	+1	-1	+1

Total ones = 4. The best subarray of b is positions 2...6 with sum $1 + 1 - 1 + 1 + 1 = 3$.

Answer

Maximum ones = $4 + 3 = 7$

Watch the edge case: if a is all ones, the move still has to happen, so the answer is (total ones) - 1.

Q16: Vitamins

A shop sells n juices. Juice i costs c_i and contains some non-empty subset of the vitamins $\{A, B, C\}$.

What is the minimum total price to obtain **all three** vitamins?

Return -1 if it is impossible.

juice	1	2	3	4
price	5	6	16	4
vitamins	C	B	BAC	A

Q16: Solution

Only the **set** of vitamins matters, and there are just $2^3 = 8$ subsets. Encode each juice as a 3-bit mask.

Definition

$dp[mask]$ = minimum cost to obtain exactly the vitamins in $mask$ (or better).

Start with $dp[000] = 0$ and everything else $+\infty$. For each juice with mask m and price c , and every s :

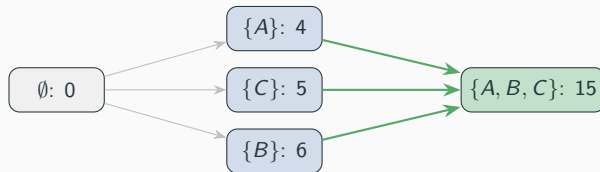
$$dp[s \mid m] \leftarrow \min(dp[s \mid m], dp[s] + c).$$

The answer is $dp[111]$, or -1 if it is still $+\infty$.

Answer

$O(n \cdot 2^3) = O(n)$ time

Q16: Example



Buying the three single-vitamin juices costs $4 + 5 + 6 = 15$, which beats the all-in-one juice at 16.

Answer

Minimum total price = 15